

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Apropiar un paquete revisando el diagrama de clases, la documentación y el código.
2. Crear y manipular un objeto. Extender y crear una clase.
3. Entender el comportamiento básico de memoria en la programación OO.
4. Investigar clases y métodos en el [API](#) de Java.
5. Utilizar el entorno de desarrollo de BlueJ.
6. Vivenciar las prácticas XP:

Planning — el proyecto se divide en iteraciones; **Coding** — todo el código de producción se programa en parejas.

ENTREGA

- Incluyan en un archivo **.zip** los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion. (CasallasC-DuarteJ)
- Publiquen el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

PARTE I. SHAPES

A Conociendo el proyecto. [En [lab01.doc](#)]

1. El proyecto [shapes](#) es una versión modificada de un recurso ofrecido por BlueJ. Para trabajar con él, bajen [shapes.zip](#) y ábralo en BlueJ¹. Capturen la pantalla.
2. El **diagrama de clases** permite visualizar las clases de un artefacto software y las relaciones entre ellas. Considerando el diagrama de clases de [shapes](#): (a) ¿Qué clases ofrece? (b) ¿Qué relaciones existen entre ellas?
3. La **documentación**² presenta las clases del proyecto y, en este caso, la especificación de sus componentes públicos. Genere la documentación. De acuerdo con la documentación: (a) ¿Qué clases tiene el paquete [shapes](#)? (b) ¿Qué atributos tiene la clase [Triangle](#)? (c) ¿Cuántos métodos ofrece la clase [Triangle](#)? (d) ¿Qué atributos determinan el tamaño de un [Triangle](#)? (e) ¿Cuáles métodos ofrece la clase [Triangle](#) para cambiar su tamaño?
4. En el **código fuente** de cada clase está el detalle de la implementación. Revisen el código de la clase [Triangle](#). Con respecto a los atributos: (a) ¿Cuántos atributos realmente tiene? (b) ¿Quiénes pueden usar los atributos públicos?. Con respecto a los métodos: (c) ¿Cuántos métodos tiene en total? (d) ¿Quiénes usan los métodos privados?
5. Desde el editor, consulte la documentación de [Triangle](#). (a) Capture la pantalla. Comparando la documentación con el código: (b) ¿Qué no se ve en la documentación? (c) ¿por qué debe ser así?
6. ¿Cuál dirían es el propósito del proyecto [shapes](#)?

B Manipulando objetos. [En [lab01.doc](#)]

1. Creen un objeto de cada una de las clases que lo permitan³ y háganlos visible. ⁴ (a) ¿Cuántas clases tiene el proyecto? (b) ¿Cuántos objetos crearon? (c) ¿Qué clase se crea de forma diferente? ¿Por qué? (d) Capturen la pantalla.
2. Inspeccionen los creadores de cada una de las clases. (a) ¿Cuál es la principal diferencia entre ellos? (b) ¿Qué se busca con la clase que tiene el creador diferente?

¹Menú: [Project](#) → [Open](#).

²Menú: [Tools](#) → [Project Documentation](#).

³Clic derecho sobre la clase

⁴Clic derecho sobre el objeto

3. Construyan, con `shapes` sin escribir código, una propuesta de la imagen del logo de su navegador favorito.
 - (a) ¿Cuántas y cuáles clases se necesitan?
 - (b) ¿Cuántos objetos se usan en total?
 - (c) Capturen la pantalla.
 - (d) Incluyan el logo original

C Detallando una clase y explorando sus objetos. [En `lab01.doc`]

1. En el código de la clase `Triangle`, revise el atributo `VERTICES`. (a) ¿Qué significa que sea `public`? (b) ¿Qué significa que sea `static`? (c) ¿Debería ser `final`? ¿Por qué? (d) Realicen los cambios necesarios.
2. En el código de la clase `Triangle` revisen el detalle del tipo del atributo `height`. (a) ¿Qué se está indicando al decir que es `int`? (b) Si fuera `byte`, ¿cuál sería el área más grande posible? (c) y ¿si fuera `long`? (d) ¿qué restricción adicional debería tener este atributo? (e) Refactoricen el código considerando (d).
3. Si creamos 100 triángulos, (a) ¿cuántos `VERTICES` y cuántos `height` tendríamos? (b) Justifique.
4. Inspeccionen el **estado** del objeto `:Triangle5`. (a) Capturen la pantalla. (b) ¿Cuál es el valor de `height` inicial? (c) ¿Cuál es el valor de `VERTICES`?
5. Inspeccionen el **comportamiento** que ofrece el objeto `:Triangle6`. (a) Capturen la pantalla. (b) ¿Por qué no aparecen todos los métodos que están en el código?

D Analizando y escribiendo código. [En `lab01.doc`]

```

Circle face;
Circle eye;
Triangle mouthUp;
Triangle mouthDown;
//1
face = new Circle();
face.changeColor("yellow");
face.changeSize(100);
//2
eye = new Circle();
eye.changeColor("black");
eye.changeSize(15);
eye.moveVertical(10);
eye.moveHorizontal(35);
eye.makeVisible();
//3

mouthUp = new Triangle();
mouthUp.changeColor("white");
mouthUp.changeSize(30, 90);
mouthUp.moveHorizontal(-20);
mouthUp.moveVertical(15);
mouthUp.makeVisible();
//4
mouthDown = new Triangle();
mouthDown.changeColor("white");
mouthDown.changeSize(-30, 90);
mouthDown.moveHorizontal(-20);
mouthDown.moveVertical(75);
mouthDown.makeVisible();
//5
Circle food=eye;
food.changeColor("red");
food.moveHorizontal(100);
food.moveVertical(25);
food.makeVisible();
//6

```

1. Lean el código anterior. (a) ¿Cuál creen que es la figura resultante? (b) Píntenla.
2. Para cada punto señalado: (a) ¿cuántas variables existen? (b) ¿cuántos objetos existen? (c) ¿cuántos objetos se ven? (no cuenten ni los objetos `String` ni el objeto `Canvas`) (3 respuestas en cada punto)
3. Habiliten la ventana de código en línea⁷, escriban el código. (a) Capturen la pantalla.
4. Compare la figura pintada en 1. con la figura capturada en 3.: (a) ¿son iguales? (b) ¿por qué?
5. (a) ¿Qué errores hacen que la figura no quede bien pintada? (b) ¿Qué figura es?

E Extendiendo una clase. `Triangle`. [En `lab01.doc`]

1. Desarrollen en `Triangle` el método `area()` (retorna el área). ¡Pruébenlo! Capturen una pantalla.
2. Desarrollen en `Triangle` el método `equilateral()` (lo convierte en un triángulo equilátero de aproximadamente la misma área) ¡Pruébenlo! Capturen dos pantallas.
3. Desarrollen en `Triangle` el método `walk(times: int)` (va moviéndose cayendo hacia la derecha (positivo) o izquierda (negativo) `abs(times)` veces). ¡Pruébenlo! Capturen tres pantallas.

⁵Clic derecho sobre el objeto y comando *Inspect*

⁶Clic derecho sobre el objeto

⁷Menú: `View` → `Show Code Pad`

4. Desarrollen en [Triangle](#) un nuevo creador que permita crearlo dado su color, su ancho y su alto. ¡Pruébenlo! Capturen una pantalla.
5. Propongan un nuevo método para esta clase. Desarrollen y prueben el método.
6. Generen nuevamente la documentación y revisen la información de estos nuevos métodos. Capturen la pantalla.

PARTE II. De Python a Java

En este punto vamos a usar y evaluar dos recursos de apoyo para la transición de Python a Java. Realicen la evaluación en las encuestas preparadas con ese objetivo:

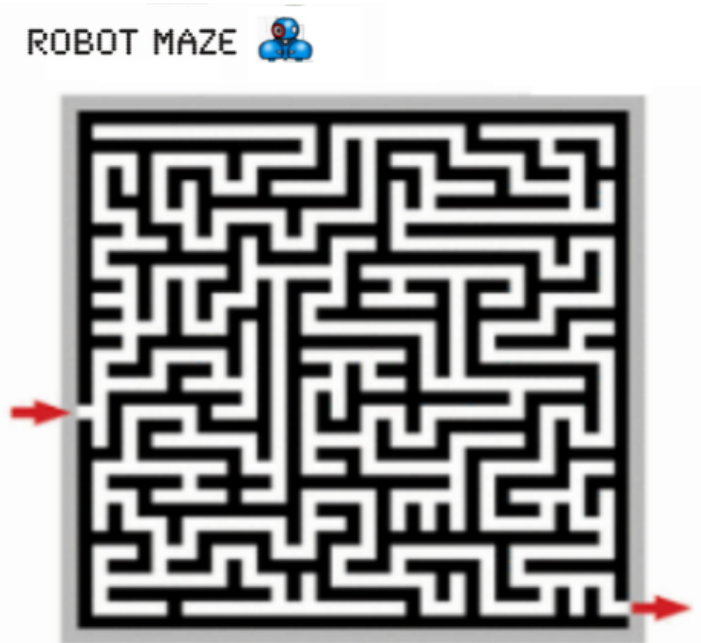
1. El video.
2. Los prompts.

PARTE III. RobotMaze

El objetivo de este punto es desarrollar una mini-aplicación para una versión simplificada del juego RobotMaze. Este juego está inspirado en Karel, un emulador de un robot, distribuido como software educativo de código abierto, creado Richard E. Pattis en su libro *Karel: The Robot: A Gentle Introduction to the Art of Programming*. [VINCULO](#)

El juego se desarrolla en un laberinto con una entrada, una salida y un conjunto de paredes. El robot inicia en la entrada y el usuario controla sus movimientos mediante comandos. El objetivo del juego es guiar al robot hasta la salida consumiendo la menor cantidad posible de vidas.

- El jugador indica el tamaño del juego. La entrada se crea en una posición aleatoria y la salida en una posición aleatoria de la cara opuesta.
- El jugador coloca las paredes en el laberinto.
- Cuando el jugador decide iniciar el juego, el robot se ubica en la entrada. A partir de este momento puede mover el robot que tiene 10 puntos de vida.
- El jugador puede pedirle al robot que gire o que avance una casilla en la dirección actual
El robot puede girar en las cuatro direcciones (North, South, East, West)
- Si el robot choca contra una pared o los bordes del laberinto, pierde 1 punto de vida.
- Si el robot se queda sin vida, no puede realizar ningún movimiento.



A Creando una nueva clase. Usando el paquete `shapes` [En `lab01.doc`]

En este punto vamos a crear la clase `Robot` del juego. El diseño gráfico lo definen ustedes.

Robot + <code>_ (x : int, y : int) : Robot</code> + <code>move(step : int) : void</code> + <code>turn(direction : char) : void</code> + <code>coordinates() : int[]</code> + <code>direction() : char</code> + <code>makeVisible() : void</code> + <code>makeInvisible() : void</code> + <code>isOK() : boolean</code>	■ Ciclo 1: - <code>_()</code> - <code>coordinates</code> - <code>direction</code> ■ Ciclo 2: - <code>move</code> - <code>turn</code> - <code>isOK</code> ■ Ciclo 3: - <code>makeVisible</code> - <code>makeInvisible</code> ■ Notas: + Puede mirar a cuatro posibles direcciones: N, S, E, W. - Se crea mirando al norte. - <code>coordinates</code> retorna la posición {x,y} - En <code>move</code>, el parámetro puede ser negativo - <code>isOK</code> retorna falso si no se pudo hacer el último movimiento
---	---

1. Inicie la construcción únicamente con los **atributos**. Justifique su selección. Agregue pantallazo con los atributos.
2. Desarrollen la clase considerando los 3 mini-ciclos. No olviden la documentación. Al final de cada mini-ciclo realicen dos pruebas indicando su propósito. Capturen las pantallas relevantes.

B Definiendo y creando una nueva clase. `RobotMaze`. [En `lab01.doc`]

En este punto vamos a desarrollar la *mini-aplicación* para `RobotMaze`.

Requisitos funcionales	Requisitos de interfaz
■ Crear el laberinto indicando su tamaño. (Ubica la entrada y la salida) ■ Adicionar paredes al laberinto. (Únicamente antes de iniciar el juego) ■ Iniciar el juego. ■ Hacer los movimientos deseados ■ Consultar la vida disponible ■ Indicar cuando se cumplen las condiciones de fin del juego: el robot alcanza la salida o el robot se queda sin vida ■ Terminar el juego, cuando se desee	■ La entrada y la salida deben diferenciarse ■ La dirección a la que mira el robot debe ser evidente ■ Las paredes con las que se golpea el robot adquieren un color diferente ■ El robot debe lucir diferente si se quedó sin vida ■ En caso que no sea posible realizar una de las acciones, debe generar un mensaje de error. ■ Para los mensajes deben usar <code>JOptionPane</code>.

1. Diseñen la clase, es decir, definan los métodos que debe ofrecer.
2. Planifiquen la construcción considerando algunos mini-ciclos.
3. Implementen la clase. Al final de cada mini-ciclo realicen una prueba indicando su propósito. Capturen las pantallas relevantes.
4. Indiquen las extensiones necesarias para reutilizar la clase `Robot` y el paquete `shapes`. Expliquen.

C BONO. Nuevos requisitos funcionales. `RobotMaze`. [En `lab01.doc`]

Extiendan la mini-aplicación `RobotMaze` con los siguientes requisitos:

- Hacer un buen movimiento (la máquina decide). Explique la estrategia.
- Deshacer el último movimiento. Deshacer se considera un movimiento.

1. Diseñen, es decir, definan los métodos que debe ofrecer.
2. Implementen los nuevos métodos. Al final de cada método realicen una prueba indicando su propósito. Capturen las pantallas relevantes.

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?
4. ¿Cuál consideran fue el mayor logro? ¿Por qué?
5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.